Programmation sur Processeur Graphique

—

Partie 8 : Considérations sur les performances : memory coalescing

Pierre-Emmanuel Hladik
pierre-emmanuel.hladik@ec-nantes.fr

11 mai 2025

Bande passante de la mémoire globale

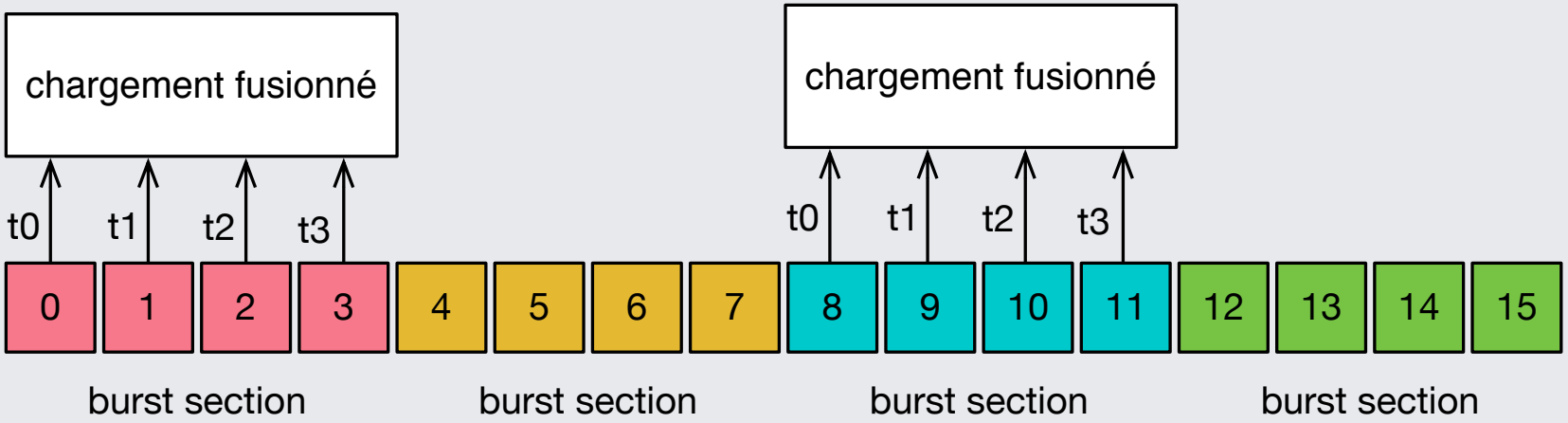
- La mémoire globale d'un dispositif CUDA est implémentée avec des DRAM
- **La DRAM est lente**

Les bits de données sont stockés dans des cellules DRAM qui sont de petits condensateurs. La quantité de charge électrique fait la différence entre 0 et 1.
Pour lire les données d'une cellule DRAM, le condensateur doit utiliser sa charge électrique pour alimenter une ligne hautement capacitive menant à un capteur et déclencher son mécanisme de détection qui détermine si la quantité de charge présente dans le condensateur est suffisante pour être considérée comme un "1". Ce processus prend 10 nanosecondes dans les puces DRAM modernes. Cela contraste fortement avec le temps de cycle d'horloge inférieur à la nanoseconde des dispositifs informatiques modernes.

DRAM burst

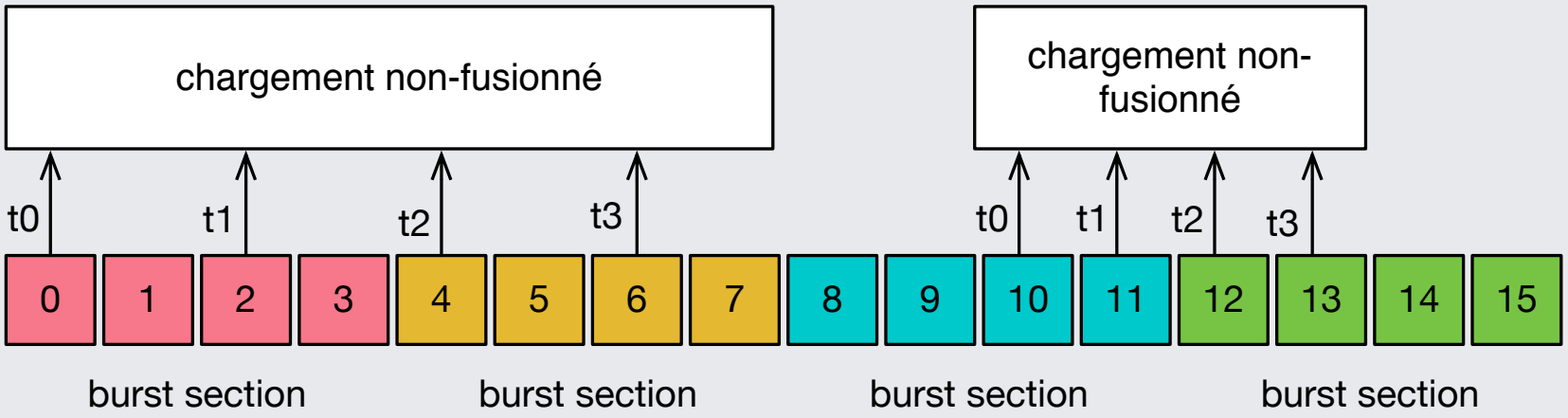
- Les DRAM modernes utilisent le parallélisme pour augmenter leur taux d'accès aux données
- Chaque fois qu'on accède à un emplacement DRAM, on accède à une série d'emplacements consécutifs comprenant l'emplacement demandé et le contenu d'emplacements proches.
- Une fois détectées les données de tous ces emplacements consécutifs sont transférées au processeur. Ces emplacements consécutifs auxquels on accède et qui sont délivrés sont appelés des **bursts** (rafales) de DRAM.

Memory Coalescing



Quand les threads exécutent une instruction de chargement, si tous les emplacements accédés tombent dans la même section du brust, une seule demande de DRAM sera faite et l'accès est entièrement fusionné (*coalesced*)

Accès Un-coalesced



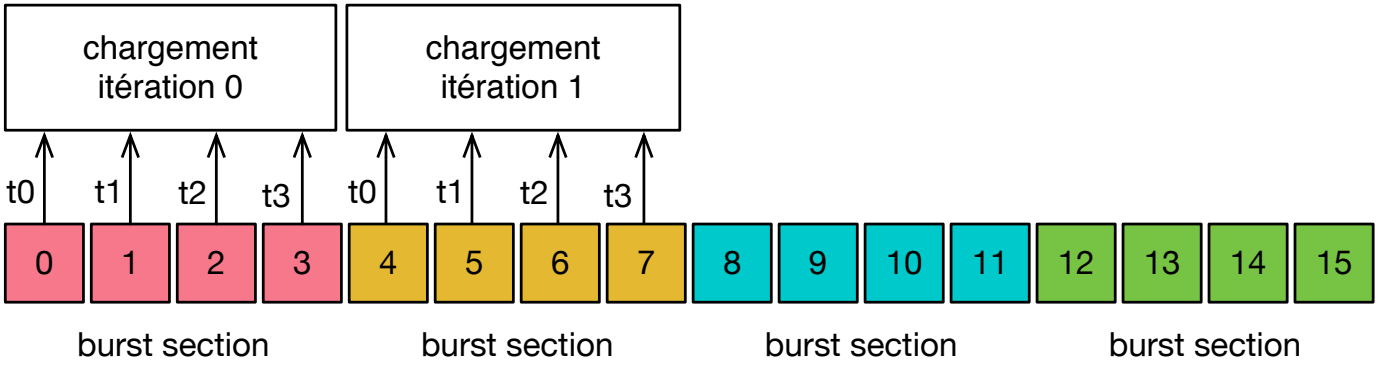
Lorsque les adresses accédées par les threads sont sur plusieurs sections de bursts :

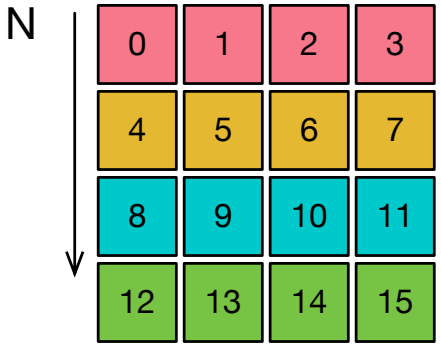
- La fusion échoue
- Plusieurs demandes de DRAM sont faites
- Certains des octets auxquels on accède et qui sont transférés ne sont pas utilisés par les threads

Exemple avec la multiplication matricielle naïve

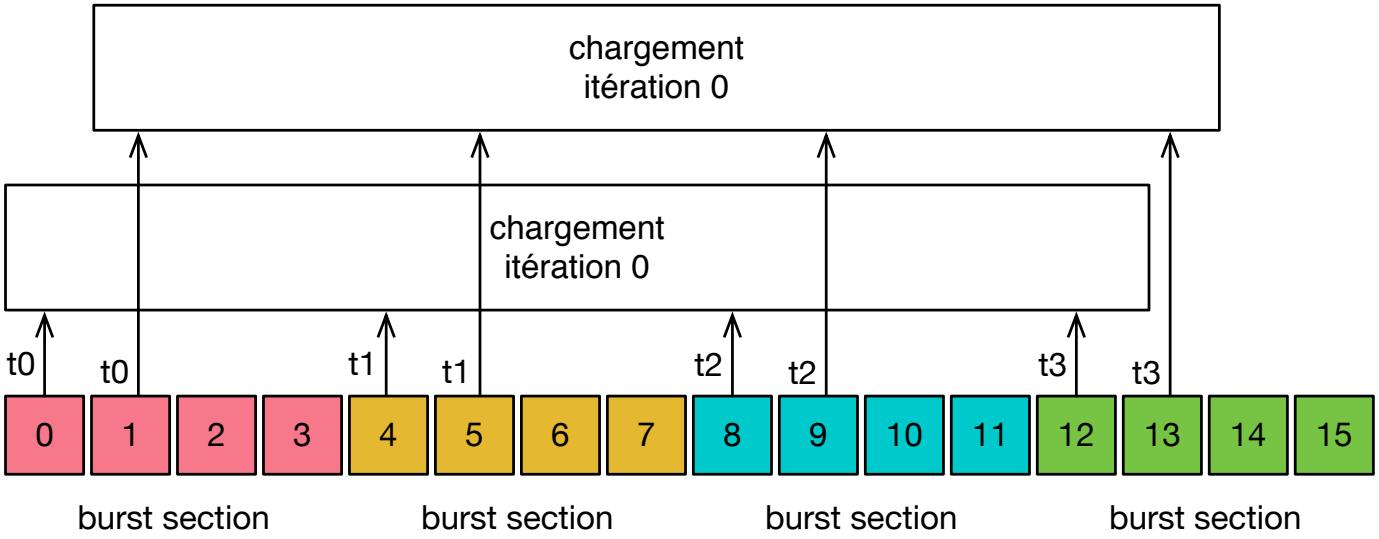
```
__global__
void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if ((row < width) && (col < width)) {
    float sum = 0;
    for (int k = 0; k < width; k++)
    {
      sum += M[row * width + k]*N[k * width + col];
    }
    P[row * width + col] = sum;
  }

}
```

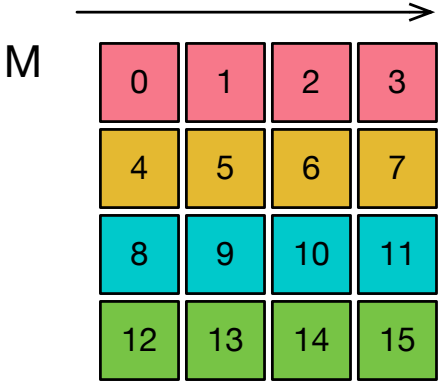
Exemple : produit matriciel – accès fusionné





Exemple : produit matriciel – accès non fusionné





Caractéristique mémoire de la Jetson Nano

Caractéristique : 2GB 64-bit LPDDR4 ; 25.6 gigabytes/second, soit :

- 2Go de mémoire LPDDR4 (Low Power Double Data Rate 4), une mémoire vive à faible consommation énergétique optimisée pour les systèmes embarqués.
- un bus mémoire de 64 bits de large, ce qui signifie que chaque accès peut transférer 64 bits (8 octets) à la fois.
- avec une bande passante mémoire maximale à 25.6 Go/seconde, qui représente la vitesse théorique à laquelle le système peut lire ou écrire des données en RAM

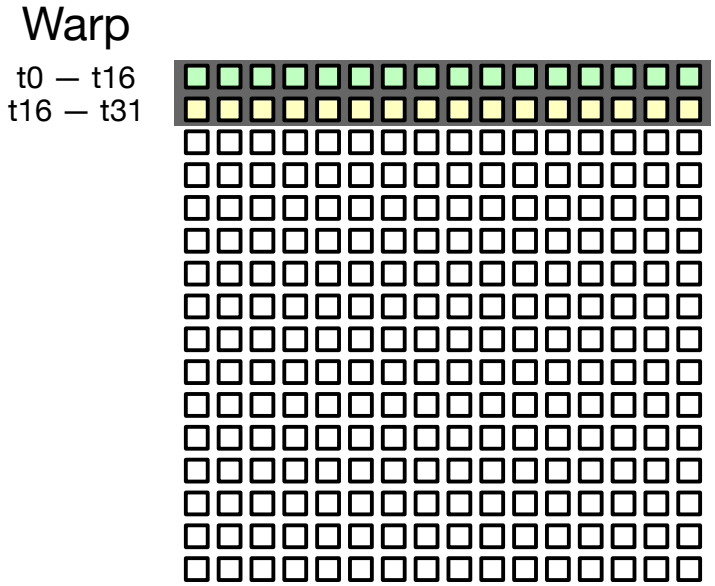
Taille du "burst"

Dans les mémoires LPDDR4, un accès en burst signifie que plusieurs mots de données sont lus ou écrits en rafale. La taille minimale d'un burst pour LPDDR4 est généralement 16 mots de 32 bits (soit 64 octets).

Exemple avec la multiplication matricielle par tuile

```
__global__
void MatrixMulKernel(float* M, float* N, float* P, int size)
{
  __shared__ float ds_M[TILE_SIZE][TILE_SIZE];
  __shared__ float ds_N[TILE_SIZE][TILE_SIZE];

  int bx = blockIdx.x; int by  = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;
  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float sum= 0.0;

  for (int p = 0; p < n/TILE_SIZE; ++p) {
    ds_M[ty][tx] = M[Row * size + p * TILE_SIZE + tx];
    ds_N[ty][tx] = N[(p * TILE_SIZE + ty) * size + Col];
    // eq.: ds_N[ty][tx] = N[(p * TILE_SIZE + ty) * size + bx * blockDim.x + tx];
    __syncthreads();

    for (int k = 0; k < TILE_SIZE; k++) {
      sum += ds_M[ty][k] * ds_N[k][tx];
    }
    __syncthreads();
  }
  P[Row*size+Col] = sum;
}
```

Exemple : produit matriciel – accès fusionné – warp



Taille d'un bloc : 16x16
Float = 32 bits (4o)
1 burst = 64o/4 = 16 floats

Un peu de pratique avec le TD 8

Objectifs

- Observer les effets d'un accès fusionnée ou non